SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 1 – Analytical Problem Solving (5 QUESTIONS)

Course Code:	MLA0407	**Course Title:**	**Deep Learning**
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – **Analytical problem solving**
Weightage:	35% of CIA	**Total Marks:**	(5 Questions × 10 Marks)
CO1 Statement:	***Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.***		
Student Name:	Om Swaroop pannuru	**Reg. No.:**	192425275
Section / Batch:	2rd year	**Date:**	17-07-26

CodeofConduct

*I om Swaroop pi(192425275)___(NAME/ REG NO) certify that this submission is my original workand thatI have adhered to the guidelinesspecifiedforthisassessment. Iunderstandthatanyviolationofacademicintegrityruleswillresultindisciplinaryaction.*Signature of the Student:om Swaroop pannuru___

QUESTIONS: 5 **Total Marks: 50**

Annexure A

Analytical Problem Solving **Duration: 1hr**

Q1.

Design a machine learning solution for predicting customer loan approval using a suitable learning algorithm. **Describe** the major stages involved in model development, including data preparation, model training, and performance evaluation. (10 marks)

Q2.

Describe the architecture of an Artificial Neural Network (ANN) consisting of input, hidden, and output layers. **Illustrate** the role of artificial neurons, weights, bias, and activation functions in processing input data to generate predictions. (10 marks)

Q3.

Analyze the working of the Perceptron learning algorithm for solving a binary classification problem. **Demonstrate** the weight and bias update process for one iteration using a suitable numerical example. (10 marks)

Q4.

Illustrate the forward propagation process in a Multilayer Perceptron (MLP) using a simple neural network with one hidden layer. **Compute** the output values for the given inputs, weights, biases, and activation function. (10 marks)

Q5.

Demonstrate the Backpropagation algorithm for training a neural network by calculating the output error, propagating the error through hidden layers, and updating the network weights using Gradient Descent. (10 marks)

Q6.

Compare and evaluate Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent with respect to convergence behavior, computational efficiency, memory requirements, and practical applications in deep learning. (10 marks)

Q7.

Formulate a Maximum Likelihood Estimation (MLE) approach for estimating the parameters of a probability distribution from a given dataset. **Interpret** the estimated parameters and discuss their significance in machine learning model development. (10 marks)

Q8.

Analyze the impact of the Curse of Dimensionality on machine learning models with respect to training complexity, prediction accuracy, and data sparsity. **Propose** suitable techniques to overcome these challenges in real-world applications. (10 marks)

Q9.

Design a Multilayer Perceptron (MLP) model for a multiclass image classification problem. **Describe** the network architecture and explain the role of forward propagation, backpropagation, and Gradient Descent during the training process. (10 marks)

Q10.

Evaluate the importance of learning algorithms in building intelligent machine learning

systems. **Compare** traditional machine learning approaches with neural network-based learning by considering model complexity, learning capability, and application scenarios. (10 marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Name : Om Swaroop

Reg. No. : 192425275

Course : MLA0404 - DEEP LEARNING

Tool : C01 – AT1 - Analytical Problem Solving

Q1. Machine Learning Solution for Customer Loan Approval

Customer loan approval prediction is a binary classification problem, where the model predicts whether a loan should be Approved or Rejected. A suitable algorithm for this problem is Logistic Regression, as it works well for binary classification and provides an interpretable probability output.

1. Data Collection and Preparation

Historical customer data is collected with features such as:

- Income
- Credit score
- Employment status
- Loan amount
- Existing debts
- Repayment history
- Loan approval status

The data is cleaned by handling missing values, removing duplicate records, and correcting inconsistencies. Categorical data, such as employment status, is converted into numerical form using encoding. The dataset is then divided into training and testing sets, usually in an 80:20 ratio.

2. Model Training

Logistic Regression calculates the probability of loan approval using the sigmoid function:

$$\text{Probability} = 1 / (1 + e^{-z})$$

where,

$$z = (w_1 \times x_1) + (w_2 \times x_2) + \ldots + (w_n \times x_n) + \text{Bias}$$

Here, x represents the input features and w represents their corresponding weights. During training, the model learns suitable weights and bias from historical data.

3. Prediction

The trained model predicts the probability of loan approval. If a threshold of 0.5 is used:

- Probability $\geq 0.5 \rightarrow$ Loan Approved
- Probability $< 0.5 \rightarrow$ Loan Rejected

4. Performance Evaluation

The model is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- ROC-AUC Score

Thus, Logistic Regression provides an effective and interpretable solution for predicting customer loan approval. The model should also be checked for fairness and bias before real-world deployment.

Q2. Architecture of an Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by biological neural networks. It consists of interconnected artificial neurons arranged into three main types of layers.

1. Input Layer

The input layer receives the features from the dataset. For example, in loan prediction, income, credit score, and loan amount can be given as inputs.

2. Hidden Layer

The hidden layer performs calculations on the input data and learns important patterns. A neural network may contain one or more hidden layers.

3. Output Layer

The output layer generates the final prediction. In binary classification, it can produce a probability between 0 and 1.

Working of an Artificial Neuron

Each neuron receives input values and multiplies them by their corresponding weights. A bias is added to the result.

Weighted Sum = $(w_1 \times x_1) + (w_2 \times x_2) + \ldots + \text{Bias}$

The result is then passed through an activation function:

Output = Activation Function (Weighted Sum)

The main components are:

- Artificial Neuron: Processes the input information.
- Weight: Determines the importance of each input.
- Bias: Adjusts the output of the neuron.
- Activation Function: Introduces non-linearity into the network.

Common activation functions include Sigmoid, ReLU, and Tanh.

Therefore, data flows through the ANN as:

Input Layer → Hidden Layer(s) → Output Layer

This process enables the network to learn patterns from training data and generate predictions.

Q3. Perceptron Learning Algorithm for Binary Classification

The Perceptron is a supervised learning algorithm used for binary classification of linearly separable data. It learns by adjusting its weights and bias whenever it makes an incorrect prediction.

The weighted sum is calculated as:

Weighted Sum = $(w_1 \times x_1) + (w_2 \times x_2) +$

Bias The step activation function is applied:

- If Weighted Sum ≥ 0, Predicted Output = 1
- If Weighted Sum < 0, Predicted Output = 0

The error is calculated as:

Error = Actual Output − Predicted Output

The weights are updated using:

New Weight = Old Weight + (Learning Rate × Error × Input)

The bias is updated using:

New Bias = Old Bias + (Learning Rate × Error)

Numerical Example

Consider:

Inputs: $x_1 = 1$, $x_2 = 2$
Actual Output: $y = 1$
Initial Weights: $w_1 = -0.2$, $w_2 = 0.1$
Bias: -0.5
Learning Rate: 0.1

Step 1: Calculate Weighted Sum

Weighted Sum = $(-0.2 \times 1) + (0.1 \times 2) - 0.5$

Weighted Sum = -0.5

Since the weighted sum is less than 0:

Predicted Output = 0

Step 2: Calculate Error

Error = $1 - 0 = 1$

Step 3: Update Weights

New $w_1 = -0.2 + (0.1 \times 1 \times 1)$
New $w_1 = -0.1$

New $w_2 = 0.1 + (0.1 \times 1 \times 2)$
New $w_2 = 0.3$

Step 4: Update Bias

New Bias = $-0.5 + (0.1 \times 1)$
New Bias = -0.4

Therefore, after one iteration:

$w_1 = -0.1$, $w_2 = 0.3$, Bias = -0.4

This process is repeated for the training samples until the Perceptron learns a suitable decision boundary.

Q4. Forward Propagation in a Multilayer Perceptron (MLP)

Forward propagation is the process of passing input data from the input layer through the hidden layer to the output layer.

Consider a simple MLP with:

- 2 input neurons
- 2 hidden neurons
- 1 output neuron
- Sigmoid activation function

The sigmoid function is:

$\text{Sigmoid}(z) = 1 / (1 + e^{-z})$

Given inputs:

$x_1 = 1, x_2 = 2$

Step 1: Calculate First Hidden Neuron (h_1)

Given:

$w_{11} = 0.5, w_{21} = 0.2, \text{Bias} = 0.1$

$z_1 = (1 \times 0.5) + (2 \times 0.2) + 0.1$

$z_1 = 1.0$

Applying Sigmoid:

$h_1 = \text{Sigmoid}(1.0) = 0.731$

Step 2: Calculate Second Hidden Neuron (h_2)

Given:

$w_{12} = 0.3, w_{22} = 0.4, \text{Bias} = 0.2$

$z_2 = (1 \times 0.3) + (2 \times 0.4) + 0.2$

$z_2 = 1.3$

Applying Sigmoid:

$h_2 = \text{Sigmoid}(1.3) = 0.786$

Step 3: Calculate Output

Output weights:

$v_1 = 0.6$, $v_2 = 0.7$, Output Bias = 0.1

Output Weighted Sum = $(0.731 \times 0.6) + (0.786 \times 0.7) + 0.1$

Output Weighted Sum = 1.089

Applying Sigmoid:

Final Output = Sigmoid(1.089)

Final Output ≈ 0.748

Therefore, the final output of the MLP after forward propagation is approximately 0.748.

Q5. Backpropagation Algorithm Using Gradient Descent

Backpropagation is used to train a neural network by calculating the prediction error and propagating it backward through the network. The weights are then updated using Gradient Descent to reduce the error.

Consider a simple network with one input neuron, one hidden neuron, and one output neuron.

Given:

Input (x) = 1
Actual Output (y) = 1
Input-to-Hidden Weight (w_1) = 0.5
Hidden-to-Output Weight (w_2) =
0.5 Learning Rate = 0.1
Bias = 0

The Sigmoid activation function is used.

Step 1: Forward Propagation

For the hidden neuron:

Hidden Weighted Sum = $1 \times 0.5 = 0.5$

Hidden Output = Sigmoid(0.5) = 0.6225

For the output neuron:

Output Weighted Sum = 0.6225 × 0.5 = 0.31125

Predicted Output = Sigmoid(0.31125) = 0.5772

Step 2: Calculate Output Error

Using the squared error function:

Error = ½ × (Actual Output − Predicted Output)²

Error = ½ × (1 − 0.5772)²

Error ≈ 0.0894

For Sigmoid activation:

Sigmoid Derivative = Output × (1 − Output)

The output delta is calculated as:

Output Delta = (Predicted Output − Actual Output) × Predicted Output × (1 − Predicted Output)

Output Delta = (0.5772 − 1) × 0.5772 × (1 − 0.5772)

Output Delta ≈ −0.1032

Step 3: Propagate Error to Hidden Layer

Hidden Delta = Output Delta × w_2 × Hidden Output × (1 − Hidden Output)

Hidden Delta = −0.1032 × 0.5 × 0.6225 × (1 − 0.6225)

Hidden Delta ≈ −0.0121

Step 4: Calculate Gradients

Gradient for w_2 = Output Delta × Hidden Output

Gradient for w_2 = −0.1032 × 0.6225 = −0.0642

Gradient for w_1 = Hidden Delta × Input

Gradient for $w_1 = -0.0121 \times 1 = -0.0121$

Step 5: Update Weights Using Gradient Descent

The weight update formula is:

New Weight = Old Weight − (Learning Rate × Gradient)

For w_2:

New $w_2 = 0.5 - [0.1 \times (-0.0642)]$

New $w_2 = 0.5064$

For w_1:

New $w_1 = 0.5 - [0.1 \times (-0.0121)]$

New $w_1 = 0.5012$

Therefore, the updated weights are:

$w_1 = 0.5012$ and $w_2 = 0.5064$

The backpropagation process can be summarized as:

Forward Propagation → Calculate Error → Propagate Error Backward → Calculate Gradients → Update Weights

This process is repeated over multiple training examples and epochs. As the weights are continuously updated, the prediction error gradually decreases and the neural network improves its performance.